

should be carefully chosen to maximise the return from model reuse across different design stages.

Tip #3: Model for embedded software generation

Embedded system software has become a key determinant of product features and its performance. It has helped organisations to achieve hardware commonality, thereby reducing the cost in the hardware and adding more and more software-based functionality. With latest developments in code generation technology, you can now generate a full-fledged production embedded code automatically from a model. The automatically generated code allows you to reuse the entire testing infrastructure built to test the model.

As suggested in Tip #2, one use of the model can be code generation. Automatically generating software may require a change in the culture of organisation, but with this, the embedded software engineer can focus more on integrating the algorithmic code. Instead of rewriting the code for a little change, he can now spend time in providing infrastructure for the same and improving the efficiency. Change in the role and responsibility of the software engineer will reflect the performance evaluation metrics.

Tip #4: Trust the model and retain its true IP

It comes quite naturally to the software engineers to fix the error or the last-minute requirement changes in the code itself rather than going through the automatic code generation and integration process. However, the moment this is done, the model and the software lose the synchronisation and the true intellectual property. When this model is reused in a different program, it may again lead to inconsistent results and algorithmic errors.

Models can become IP of the organisation and significantly benefit from their reuse in multiple projects. In or-

der to preserve and leverage from the IP, the organisational leadership need to emphasise on the 'right' practice for model-based design. The organisation has to enforce discipline and rigour to use and make models as the sole design language for embedded software.

Tip #5: Learn from the transition, don't outsource

In an organisation, it's a common practice to outsource the transition work to a third party. However, transitioning from the conventional development process to model-based design can be an excellent learning opportunity. It can be used to learn about the organisation and its long-term goals. It also brings out the fundamentals of current processes and their strengths and weaknesses. When this transition is performed in-house, it helps in effective implementation of model-based designs in critical areas.

Sometimes, the organisation may tend to just flip its entire process and product to model-based design. This may lead to unwanted changes if the current process is more established and performs efficiently, or if the process elements are likely to phase out in subsequent upgradation. A smarter way to do is to start with the processes and products where model-based design is definitely going to benefit in the first iteration and then the rest may follow. Completely new features should be chosen first to model. Next, move to the most fluctuating area where the design is constantly changing. Finally, model-based designs can be applied to stable portions in the development process and products of the organisation.

Tip #6: Understand the new role of software developers

Sometimes embedded software developers fear that model-based approach will replace their share of work. However, the software engineer becomes part of model-based design by structuring the software in the beginning of the

process and elaborating the executable specifications to enable code generation at the implementation phase.

The organisation and the engineer need to adopt the new role, wherein the software engineer develops the architecture to allow the legacy code to integrate with every new feature. They also need to ensure the workability of the code across various platforms and the targeted processor.

Tip #7: Model-based effects in processes and metrics

Transition to model-based design is incomplete if it is not integrated in every development process and evaluation metric. The existing processes such as style guides, configuration management, requirement management and verification processes need modifications to incorporate model-based design. This can be done with a detailed and continual training and implementation.

Performance metric also needs revision to accommodate the changes in model-based design. In conventional development, the number of lines of the code is one of the parameters to assess the progress of the project; this cannot be used in the model-based approach as the code is generated by just a click of a button.

Tip #8: Appoint a leader

An organisation will have diverse mindsets who may fear or welcome the transition to a new design process. So there arises the need of an influencer and a dictator who has proven track record of structuring an organisation. The person in this position needs to welcome the new way of doing the business and to pacify the doubts and fears. He should be the one who is looked upon by his peers and subordinates in the organisation.

Another quality required for this position is that the person should have some sort of budgetary control. Transitioning to model-based approach could be a substantial investment in